

Validaciones y arquitectura MVVM — InternetBankingApp

1. Validaciones implementadas y su justificación

Todas las validaciones se ejecutan en el ViewModel antes de guardar (método privado Validar()), y cada campo tiene su propia propiedad de error (NombreError, MontoError, etc.) que la vista muestra como un Label rojo debajo del campo. Esto permite que **varios errores se muestren a la vez, en línea**, en lugar de una alerta genérica: el usuario ve exactamente qué campos corregir y no pierde lo que ya escribió.

Login (LoginViewModel)

Validación	Por qué
Usuario obligatorio	Sin usuario no hay a quién autenticar.
Contraseña obligatoria y de al menos 4 caracteres	Evita envíos vacíos y descarta de entrada contraseñas imposibles, antes de consultar el servicio de autenticación.
Credenciales correctas (AuthService)	Mensaje general "Usuario o contraseña incorrectos" que no revela cuál de los dos falló.

Apertura de cuenta (CuentasViewModel)

Validación	Por qué
Tipo de cuenta seleccionado	El tipo (Ahorro/Corriente) define el producto; no puede inferirse.
Saldo inicial obligatorio y numérico (decimal.TryParse con cultura invariante)	El texto del Entry debe convertirse a decimal de forma segura; sin TryParse un texto como "abc" rompería la app.

Validación	Por qué
Saldo no negativo y mínimo RD\$100.00	Regla de negocio: un banco no abre cuentas con saldo negativo ni por debajo del monto mínimo de apertura.

Solicitud de préstamo (PrestamosViewModel)

Validación	Por qué
Producto obligatorio, mínimo 3 caracteres	Evita descripciones vacías o sin significado ("a").
Monto obligatorio, numérico, mayor a cero y hasta RD\$1,000,000.00	Un préstamo de 0 o negativo no tiene sentido, y el tope simula el límite de aprobación automática del banco.
Plazo obligatorio, entero, entre 6 y 360 meses	El plazo se usa como int (meses); el rango representa los plazos reales que ofrece un banco (medio año a 30 años).

Beneficiarios (BeneficiariosViewModel)

Validación	Por qué
Nombre obligatorio, mínimo 3 caracteres, solo letras y espacios	Es un nombre de persona: se rechazan dígitos y símbolos ("Juan123").
Número de cuenta obligatorio, solo dígitos, entre 8 y 12	Formato típico de un número de cuenta bancaria; evita letras o longitudes imposibles.
Número de cuenta no duplicado	Regla de integridad: no tiene sentido registrar dos veces el mismo beneficiario; se compara contra la colección existente.

Validación	Por qué
Banco obligatorio, mínimo 3 caracteres	Se necesita saber a qué entidad se enviaría la transferencia.

Transferencias (TransferenciasViewModel)

Validación	Por qué
Cuenta de origen y beneficiario destino seleccionados	Una transferencia sin origen o destino es imposible de ejecutar.
Concepto obligatorio, mínimo 3 caracteres	Deja rastro auditable del motivo del movimiento.
Monto obligatorio, numérico y mayor a cero	Mismo criterio de conversión segura que en los demás formularios.
Fondos suficientes (monto $\leq$ saldo de la cuenta origen)	Regla de negocio central: no se permite sobregirar. Se valida antes de enviar y también en BankingDataService.RegistrarTransferencia, que devuelve false si el saldo cambió — defensa en dos capas.
Requisito previo: tener al menos una cuenta y un beneficiario	El formulario ni siquiera se habilita si no existen ambos, guiando al usuario al paso que le falta.

Criterios comunes: los textos numéricos se convierten siempre con decimal.TryParse/int.TryParse (nunca Parse directo sobre entrada del usuario), se hace Trim() antes de validar y guardar, y las validaciones se ordenan de lo general a lo específico (obligatorio → formato → rango → regla de negocio), de modo que el usuario recibe el mensaje más útil primero.

2. Cómo MVVM ayudó a separar la lógica de la interfaz

La app usa el patrón **Model–View–ViewModel (MVVM)** con CommunityToolkit.Mvvm:

- **Model** (Cuenta, Prestamo, Beneficiario, Transferencia): solo datos, sin lógica de pantalla.
- **View** (páginas XAML): solo estructura visual y *bindings*. Los code-behind quedan casi vacíos (InitializeComponent y asignar BindingContext); no contienen ninguna regla de negocio.
- **ViewModel** (uno por página): estado de la pantalla, validaciones y comandos. No conoce controles ni páginas; expone propiedades observables y la vista se suscribe a ellas.
- **Services** (AuthService, BankingDataService): autenticación y datos compartidos, inyectados por constructor (inyección de dependencias registrada en MauiProgram), de modo que todas las páginas ven las mismas colecciones.

Beneficios concretos que se aprovecharon:

1. **Validación sin tocar la UI.** Validar() solo asigna cadenas a propiedades como SaldoError. El ViewModel nunca hace label.Text = ... ni label.IsVisible = ...; es el *binding* (Text="{Binding SaldoError}" + un convertidor para IsVisible) quien refleja el error en pantalla. La misma lógica funcionaría igual en Android, iOS o Windows.
2. **Comandos en lugar de manejadores de eventos.** Los botones usan Command="{Binding SolicitarCommand}" generado por [RelayCommand]. La acción de guardar vive en el ViewModel y podría dispararse desde cualquier control sin duplicar código.
3. **Actualización automática de las listas.** Las colecciones son ObservableCollection; al agregar una cuenta o beneficiario, el CollectionView se refresca solo. No hay código que "repinte" la lista manualmente.
4. **Estados de la pantalla como datos.** Mostrar/ocultar el formulario (IsFormVisible), el texto del botón ("+ Solicitar cuenta"/"Cancelar"), o el indicador de espera (IsBusy mientras se simula la autorización del banco) son simples propiedades booleanas; la vista reacciona por binding.
5. **Testabilidad y mantenimiento.** Como los ViewModels no dependen de ningún control visual, se pueden probar de forma aislada (por ejemplo, verificar que un saldo de 50 produce SaldoError) y un cambio de diseño en XAML no obliga a tocar la lógica, ni viceversa.

En resumen, MVVM permitió que las reglas del banco (mínimos, rangos, duplicados, fondos) vivan en un solo lugar comprobable, mientras el XAML se limita a describir cómo se ve cada pantalla.